

SYSTEM DESIGN

RIDE BOOKING SYSTEM



(Uber / Ola / Rapido)

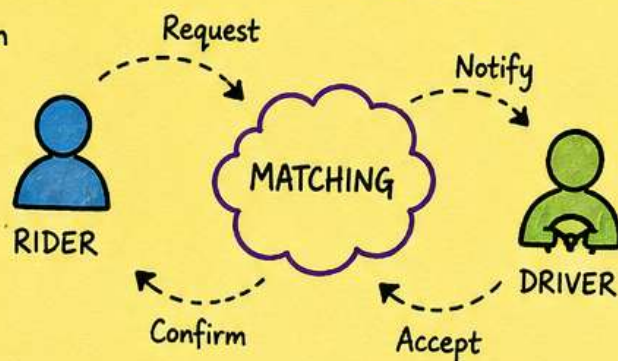
1. WHAT IS IT?

A Ride Booking System allows users to book a ride from their location to destination, get matched with a nearby driver, track live, make payments and get to their destination safely.



2. FUNCTIONAL REQUIREMENTS

- ★ User (Rider) registration & login
- ★ Search & set pickup and drop location
- ★ View fare estimate
- ★ Book / Schedule / Cancel a ride
- ★ Find nearby drivers & match
- ★ Live tracking of driver & ride
- ★ Multiple payment options
- ★ Ride history and invoices
- ★ Rate & review driver



3. NON-FUNCTIONAL REQUIREMENTS

- ★ High Availability (99.99%)
- ★ Scalability (Millions of users)
- ★ Low Latency (Real-time tracking & matching)
- ★ Data Consistency & Reliability
- ★ Security & Privacy
- ★ Fault Tolerance



4. SCALE ESTIMATION

- Users : 10M+
- Rides per day : 100M+
- Peak concurrent requests : 100K+
- Location updates/sec : Millions



5. MAJOR COMPONENTS

- ✓ Rider App
- ✓ Driver App
- ✓ Backend Services
- ✓ Databases
- ✓ Event Queue (Kafka)
- ✓ Third-party Services



♥ Good design is about people, not just systems. ♥

HIGH LEVEL ARCHITECTURE



(HLD) - Ride Booking System

RIDER APP



- Book Ride
- Live Tracking
- Payments
- History
- Reviews

DRIVER APP



- Go Online
- Receive Requests
- Navigate
- Trip Info
- Earnings

WEB DASHBOARD



- Admin Panel
- Analytics
- Operations
- Monitoring



API GATEWAY

Routing • Authentication • Rate Limiting • Load Balancing

BACKEND SERVICES (MICROSERVICES)

USER SERVICE



Riders, Drivers
Profiles, Auth

RIDE SERVICE



Create Ride,
Update, Status

MATCHING SERVICE



Find Nearby Drivers
& Match

PRICING SERVICE



Fare Calculation
& Surge

PAYMENT SERVICE



Payments, Refunds
Transactions

NOTIFICATION SERVICE



Push, SMS, Email
Alerts

LOCATION SERVICE



Track Driver &
Rider Location

RATING SERVICE



Ratings, Reviews
Feedback

TRIP HISTORY SERVICE



Trip Logs, History
Invoices

ADMIN SERVICE



Admin Ops,
Config, Rules

DATABASES



MySQL /
PostgreSQL
(Relational DB)

CACHE



Redis
(Caching,
Sessions)

EVENT QUEUE



Kafka
(Async Events,
Decoupling)

OBJECT STORAGE



S3 / MinIO
(Docs, Receipts,
Media)

SEARCH / GEO



Elasticsearch
/ GeoSearch
(Nearby Search)

THIRD PARTY



Maps (Google)
SMS / Email
Payment Gateway



Flow: Client Apps → API Gateway → Microservices → Databases / Queues / Third Party Services → Response

MICROSERVICES BREAKDOWN



(Ride Booking System)

- 1



USER SERVICE
(RIDER)

: Manages rider registration, profile, authentication, preferences, and user related operations.
- 2



DRIVER SERVICE
(DRIVER)

: Handles driver registration, documents, vehicle info, availability status, earnings and driver profile.
- 3



RIDE SERVICE
(TRIP MANAGEMENT)

: Manages ride lifecycle – create, update, start, end, cancel, status tracking, ETA, distance, duration.
- 4



MATCHING SERVICE
(DISPATCH)

: Finds nearby drivers, scores & ranks them, sends requests and assigns the best driver.
- 5



PRICING SERVICE
(FARE & SURGE)

: Calculates base fare, distance/time cost, surge pricing, discounts, coupons and final amount.
- 6



PAYMENT SERVICE
(TRANSACTIONS)

: Handles payments via UPI, Cards, Wallets, Cash, refunds, settlements and transaction history.
- 7



NOTIFICATION SERVICE
(COMMUNICATION)

: Sends push notifications, SMS, emails, in-app alerts, OTPs and updates to riders & drivers.
- 8



RATING SERVICE
(FEEDBACK)

: Manages ratings, reviews, feedback for riders and drivers, average rating calculation.
- 9



TRIP HISTORY SERVICE
(HISTORY & INVOICES)

: Stores trip history, invoices, receipts and allows users to view past rides and details.

HOW
THEY
WORK
TOGETHER


RIDER APP


API
GATEWAY


MICROSERVICES


DATASTORES


NOTIFICATIONS



Microservices are independent, loosely coupled and communicate through **APIs / Events** for better scalability and resilience.

DATABASE DESIGN

(Ride Booking System)



1. USERS 
- id (PK) - name - email (Unique) - phone (Unique) - password_hash - created_at - updated_at

Riders info

2. DRIVERS 
- id (PK) - user_id (FK) - license_no (Unique) - is_active (Bool) - rating - created_at - updated_at

Driver profile

3. VEHICLES 
- id (PK) - driver_id (FK) - vehicle_no (Unique) - vehicle_type - capacity - is_active (Bool)

Driver's vehicle(s)

4. RIDES 
- id (PK) - rider_id (FK) - driver_id (FK) - vehicle_id (FK) - pickup_location - drop_location - status - pickup_time - drop_time - distance - duration - fare_id (FK) - created_at - updated_at

STATUS

- REQUESTED
- ACCEPTED
- ONGOING
- COMPLETED
- CANCELLED

5. PAYMENTS 
- id (PK) - ride_id (FK) - user_id (FK) - amount - method (UPI/Card/Cash/Wallet) - status - transaction_id - paid_at - created_at

6. DRIVER_LOCATIONS 
- id (PK) - driver_id (FK) - latitude - longitude - updated_at

Stores real-time driver location

7. RATINGS 
- id (PK) - ride_id (FK) - rider_id (FK) - driver_id (FK) - rating (1-5) - review - created_at

Ratings & reviews

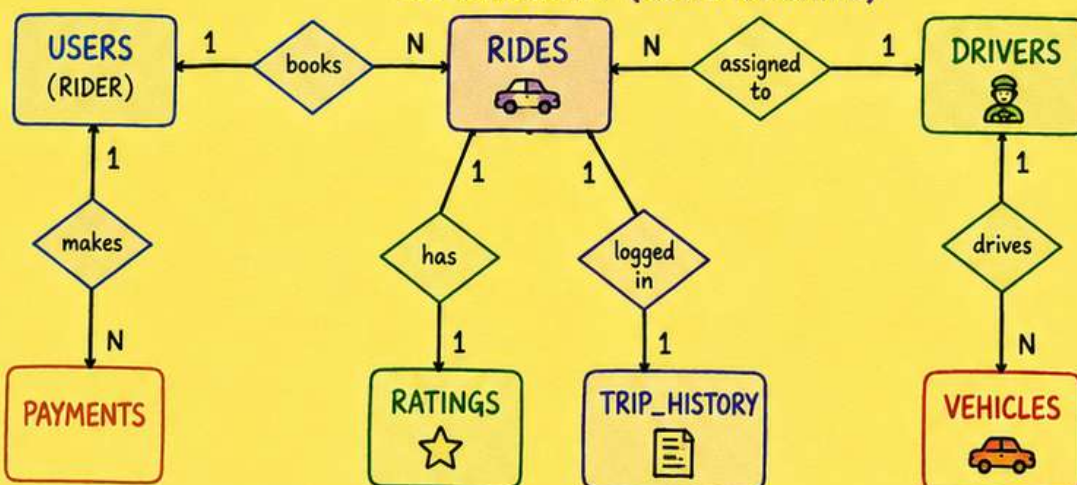
8. TRIP_HISTORY 
- id (PK) - ride_id (FK) - rider_id (FK) - driver_id (FK) - from_location - to_location - start_time - end_time - total_fare

Detailed trip logs

COMMON FIELDS

- created_at
 - updated_at
 - deleted_at
- (For soft delete & auditing)

ER DIAGRAM (RELATIONSHIPS)



RELATIONSHIP NOTES

- User (Rider) can book many Rides.
- Ride is assigned to one Driver.
- Driver can have many Vehicles.
- Ride can have one Payment.
- Each Ride can have one Rating.
- Trip History stores detailed logs for each completed ride.



INDEXING SUGGESTIONS

- users (email, phone)
- drivers (is_active)
- rides (status, created_at)
- driver_locations (driver_id, updated_at)
- payments (ride_id, status)



Well designed DB ensures fast matching, tracking, payments & analytics!

RIDE BOOKING FLOW

(STEP-BY-STEP FLOW)



1

RIDER REQUESTS RIDE



- Rider opens app
- Enters pickup & drop location
- System shows estimated fare
- Rider confirms the request

KEY DATA

- Rider ID
- Pickup, Drop
- Time, Fare Type
- Vehicle Type

2

SEARCH FOR NEARBY DRIVERS



- System finds nearby active drivers
- Using location index (GeoHash/H3)
- Filters by vehicle type & availability
- Sends ride request to shortlisted drivers

OUTPUT

- List of nearby drivers
- ETA to pickup

3

MATCHING DRIVERS



- Drivers receive ride request
- Drivers accept / ignore
- System picks the best match
- Based on ETA, rating, vehicle, etc.

MATCH FACTORS

- ETA
- Rating
- Acceptance Rate
- Distance

4

DRIVER ACCEPTS RIDE



- Selected driver accepts the ride
- Rider gets driver details
- Driver navigates to pickup location
- Rider can track live location

INFO SHARED

- Driver Name
- Vehicle No.
- ETA to Pickup
- Live Location

5

RIDE STARTS



- Driver reaches pickup location
- Rider & driver start the trip
- Trip status changes to "ONGOING"
- Meter starts (time & distance)

STATUS

- ONGOING

6

RIDE ENDS



- Driver reaches drop location
- Trip ends
- Final fare is calculated
- Rider can rate the driver

STATUS

- COMPLETED

7

PAYMENT



- Rider pays via chosen method
- Cash / UPI / Card / Wallet
- Payment is processed securely
- Invoice is generated

STATUS

- PAID

8

RECEIPT & TRIP SUMMARY



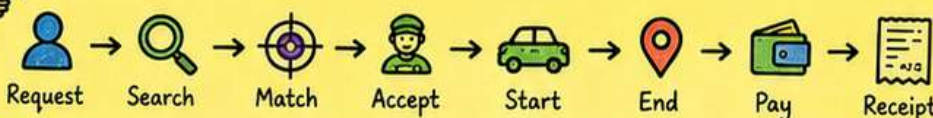
- Rider gets trip summary
- Includes distance, time, fare breakup
- Receipt sent via app / email
- Trip stored in history

RESULT

- Trip Saved
- Receipt Generated



FLOW SUMMARY



DESIGN GOALS

- Fast Matching
- Real-time Tracking
- Secure Payments
- Great User Experience

DRIVER MATCHING ALGORITHM

(How we find the best driver for a ride)



1 NEARBY SEARCH

- Get rider's current location (lat, long)
- Search for nearby active drivers
- Initial radius R (e.g., 2 km)



2 SPATIAL INDEXING

- Use GeoHash / H3 Index / QuadTree
- Helps in fast lookup of nearby drivers
- Reduces search space drastically

GeoHash Grid



WHY GEOHASH / H3?

- Covers planet in small cells
- Nearby areas have similar prefixes
- Fast range queries
- Scalable for millions of drivers



3 RADIUS EXPANSION

- If no drivers found, expand radius gradually (R, 2R, 3R ...)
- Repeat search until enough drivers are found or max radius reached



4 ETA CALCULATION

- For each driver, calculate ETA to reach rider's pickup location
- Use road distance & traffic data
- $ETA = \text{distance} / \text{avg_speed (traffic)}$



ETA = 7 min Distance = 3.2 km

5 DRIVER RANKING

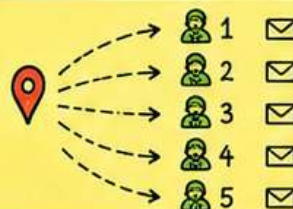
- Rank drivers based on multiple factors
- Lower score means better rank

$$\text{Score} = w_1 * \text{ETA} + w_2 * (1 - \text{Rating}) + w_3 * \text{Distance} + w_4 * \text{AcceptanceRate}$$

$$(w_1 + w_2 + w_3 + w_4 = 1)$$

6 BEST DRIVER SELECTION

- Select top N drivers (e.g., top 5)
- Send ride request to these drivers in parallel



7 ACCEPTANCE & ASSIGNMENT

- First driver who accepts gets assigned
- Others requests are cancelled
- If no one accepts within T seconds, repeat from step 2

Driver 3	ACCEPTED	✓
Driver 1	IGNORED	✗
Driver 2	IGNORED	✗



Driver 3 assigned

MATCHING PROCESS



RANKING FACTORS

- 🕒 ETA (Estimated Time)
- ★ Driver Rating
- 📍 Distance to Rider
- ✅ Acceptance Rate
- 🚗 Vehicle Type Match
- 👤 Ride Completion %
- 🚫 Cancellation Rate

DESIGN NOTES

- Matching service must be ultra low latency (< 500ms)
- Use caching for driver locations (Redis)
- Drivers send heartbeat every few seconds
- Handle driver offline / no response gracefully



KEY TAKEAWAY

Efficient indexing + smart ranking + parallel requests + fast assignment = Great matching experience!

LOCATION TRACKING SYSTEM

(Real-time Tracking of Riders & Drivers)



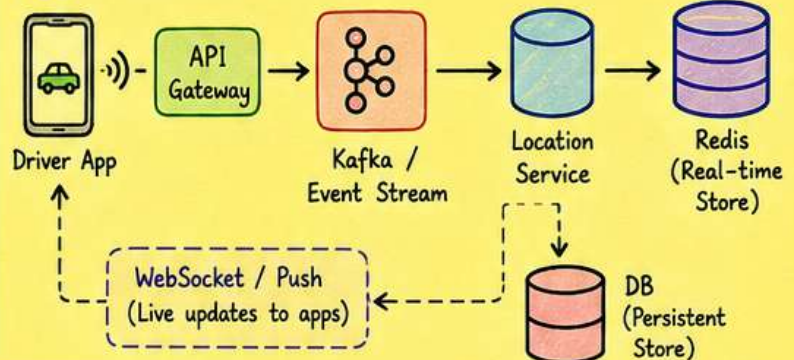
1 GPS UPDATES

- Driver app sends GPS location at regular intervals (e.g. every 5 sec)
- Contains: driver_id, lat, long, timestamp, speed
- Only active trips / online drivers send updates



Driver App

2 DATA FLOW (HIGH LEVEL)



3 TECHNOLOGY STACK

- GPS from Driver Mobile App
- API Gateway (Auth, Rate limit)
- Kafka / RabbitMQ (Event Queue)
- Location Service (Consume & Process)
- Redis (Store latest location)
- WebSocket / MQTT (Real-time push)
- DB (MySQL / Cassandra) for history

4 LIVE TRACKING FLOW

- ① Driver app gets GPS location
- ② Sends location to server
- ③ Server publishes event to Kafka
- ④ Location service consumes event
- ⑤ Updates location in Redis
- ⑥ Pushes update to rider app via WebSocket



5 REDIS DATA STRUCTURE

- Store latest location of all online drivers
- Key: driver_location:{driver_id}
- Value: {lat, long, timestamp, speed, heading}
- TTL: 30 sec (auto expire if driver offline)



redis

6 WEB SOCKET COMMUNICATION

- Used for real-time location updates
- Connection established when ride starts
- Server pushes location updates instantly
- If disconnected → auto reconnect with last known location



7 ROUTE & MAP INTEGRATION

- Use Google Maps / Mapbox API
- Show real-time driver movement
- Polyline for route
- Recalculate route if deviation > threshold



8 HANDLE OFFLINE / NO GPS

- If no location update in X seconds (e.g. 20 sec) → mark driver offline
- Show last known location
- Notify rider in extreme cases



LOCATION UPDATE PAYLOAD (Example)

```
{
  "driver_id": "D12345",
  "lat": 12.9716,
  "long": 77.5946,
  "speed": 38,
  "heading": 120,
  "timestamp": 1715856789
}
```

lat, long → location
speed → km/h
heading → direction
timestamp → epoch time



KEY POINTS

- ✓ Real-time tracking improves trust & experience
- ✓ Low latency is critical (< 500ms)
- ✓ Use caching (Redis) for fast reads
- ✓ Scalable design using Kafka + Microservices
- ✓ Fault tolerant with retries & reconnections



GOAL: Accurate, Real-time, Scalable & Reliable Location Tracking for Seamless Ride Experience! ❤️

SURGE PRICING SYSTEM

(Dynamic Pricing Based on Demand & Supply)



1 WHY SURGE PRICING?

- Balances demand & supply
- More drivers on road
- Reduce rider wait time
- Maximize platform efficiency



EXAMPLE

Normal Fare = ₹100
High Demand Multiplier = 2.5x
Surge Fare = ₹250



2 FACTORS CONSIDERED

- High Demand (More ride requests)
- Low Supply (Less available drivers)
- Time (Peak hours / Events / Weather)
- Location (Hotspots / Busy areas)
- Historical Data & Trends

3 SURGE PRICING FORMULA

$$\text{Surge Multiplier} = 1 + \alpha \left(\frac{D}{S} - 1 \right) + \beta \cdot T + \gamma \cdot L$$

Where,

D = Current Demand (ongoing ride requests)

S = Current Supply (available drivers)

T = Time Factor (peak hours, events)

L = Location Factor (hotspot weightage)

α, β, γ = Tuning Constants

MIN & MAX LIMIT

$$1x \leq \text{Multiplier} \leq 5x$$

4 SURGE ZONES (HEATMAP)

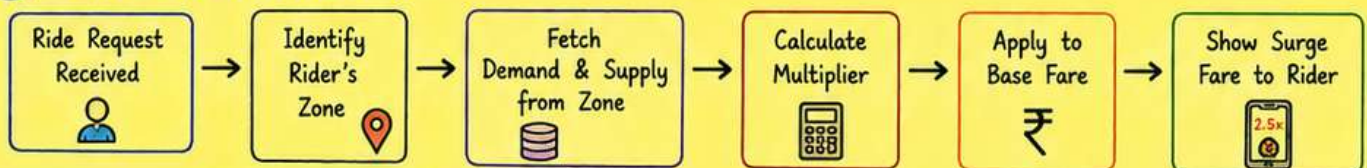
- City divided into small zones
- Each zone has dynamic multiplier
- Heatmap shown in rider app

1.1x	1.3x	1.6x
1.2x	2.5x	1.4x
1.0x	1.2x	1.1x

Higher the demand in a zone, higher the multiplier!



5 SURGE CALCULATION FLOW



6 PREVENTING SURGE ABUSE

- Cap multiplier (e.g. max 5x)
- Monitor unusual demand spikes (bot / fake requests)
- Detect & block fake locations
- Use ML models to predict real demand
- Audit logs & real-time monitoring

7 BENEFITS

- Reduced rider wait time
- More drivers on road
- Better earnings for drivers
- Efficient demand management
- Maximizes platform revenue



WIN-WIN-WIN

★ KEY POINTS

- Dynamic pricing = Fair pricing based on current conditions.
- Keeps the marketplace balanced.
- Transparent to users.
- Continuously updated every few minutes.



GOAL: Right Price, Right Time, Right Place = Happy Riders + Happy Drivers ❤️

PAYMENT ARCHITECTURE

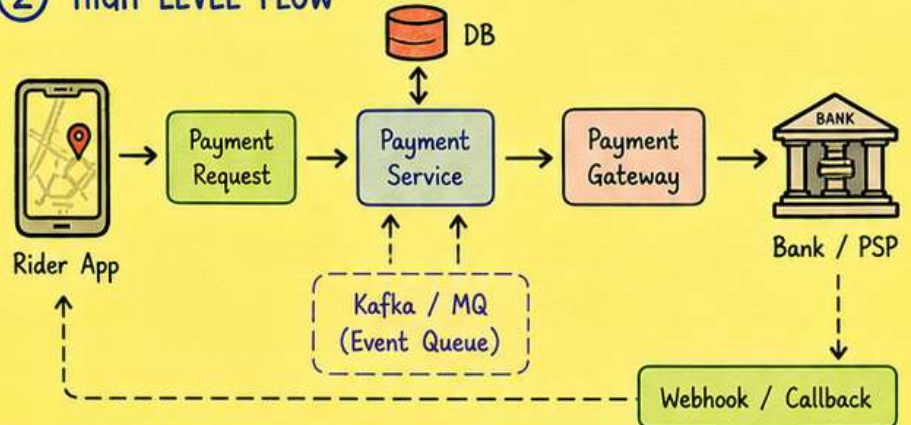
(Secure, Reliable & Scalable)



1 PAYMENT METHODS

- Cash
- UPI (GPay, PhonePe, Paytm)
- Cards (Credit / Debit)
- Wallet (In-App Wallet)
- Promo / Coupons
- Corporate / Business Accounts

2 HIGH LEVEL FLOW



3 PAYMENT MODES FLOW

- Cash → Mark as Cash → Trip Completed
- UPI / Card / Wallet
 - Create Payment Intent
 - Redirect / Collect Payment
 - Receive Status (Success / Fail)
- Corporate → Invoice Generated → Postpaid Settlement

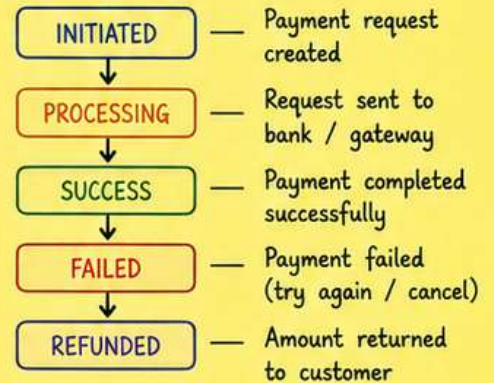
PAYMENT STATUS

- INITIATED
- PROCESSING
- SUCCESS
- FAILED
- REFUNDED



Always use Idempotency Key to avoid duplicate payments.

4 PAYMENT STATES



5 REFUND FLOW



REFUND SCENARIOS

- Rider cancels (before driver accept)
- Driver cancels (after accept)
- Payment failure but amount debited
- Overcharge / Incorrect fare

★ Refund amount depends on cancellation policy & timing.



6 IDEMPOTENCY

- Use Idempotency Key for every payment request
- Same key + same request → same response
- Prevents duplicate charge on retries / network failure

EXAMPLE HEADER

Idempotency-Key: a1b2c3d4-e5f6-7890-1234-xyz

7 SECURITY MEASURES

- PCI-DSS compliant gateways
- Tokenization for card details
- HTTPS everywhere
- Encrypt sensitive data
- Fraud detection & anomaly checks
- 2FA / OTP for high value transactions



8 ARCHITECTURE COMPONENTS

- Payment Service → Core logic
- Payment Gateway → Razorpay / Stripe / PayU
- Database → Store transactions
- Redis → Cache payment status / keys
- Kafka → Async events (payment.success, refund.initiated)
- Notification Service → Send receipts / alerts

9 KEY POINTS

- ★ Fast & Secure transactions
- ★ Handle failures gracefully
- ★ Retry with exponential backoff
- ★ Keep user informed in real-time
- ★ Always reconcile with bank
- ★ Logs & audit for every txn



GOAL: Seamless payments for every ride with security, reliability & trust! ❤️

TECHNOLOGIES

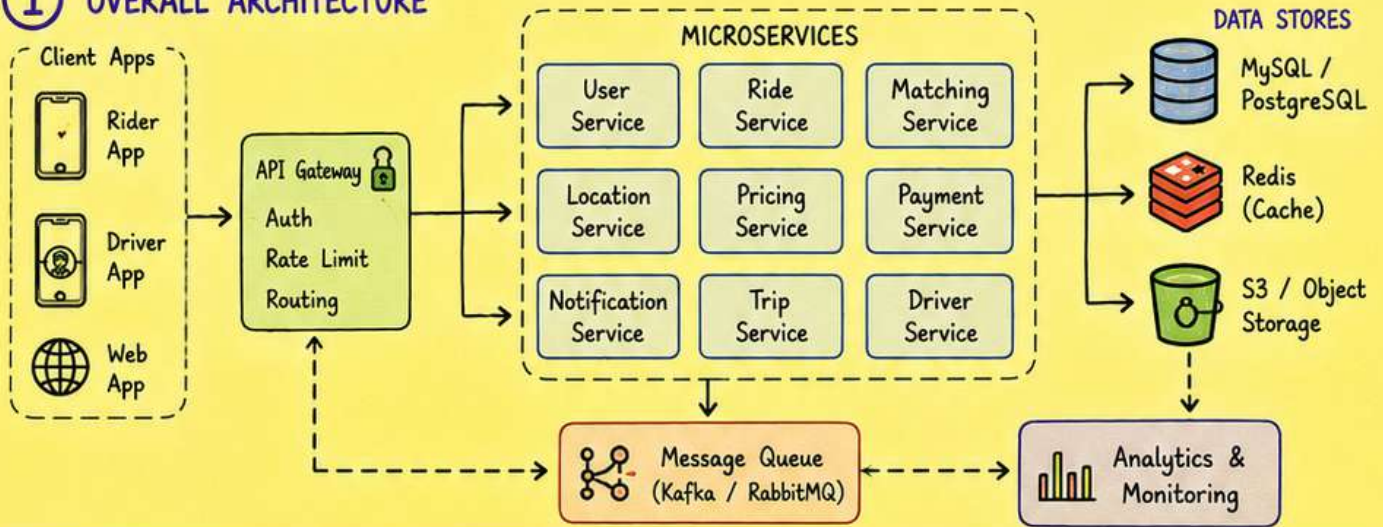
Java / Kotlin / Node.js, PostgreSQL / MySQL, Redis, Kafka, Razorpay / Stripe, Webhooks

SYSTEM ARCHITECTURE (HLD)

Scalable, Reliable & Fault-Tolerant



1 OVERALL ARCHITECTURE



2 KEY COMPONENTS

- ★ API Gateway - Single entry point for all requests
- ★ Microservices - Each service has single responsibility
- ★ Database - Persistent storage for core data
- ★ Redis - Caching, sessions, real-time data
- ★ Kafka / MQ - Async communication between services
- ★ Object Storage - Store documents, images, receipts
- ★ Analytics & Monitoring - Logs, metrics, alerts

3 TECHNOLOGY STACK

- Backend : Java / Kotlin / Node.js
- Database : MySQL / PostgreSQL
- Cache : Redis
- Message Queue : Kafka / RabbitMQ
- Storage : AWS S3 / GCP Cloud Storage
- Infra : Docker, Kubernetes
- Monitoring : Prometheus, Grafana

WHY MICROSERVICES?

- ✓ Scalability
- ✓ Independent Deployment
- ✓ Fault Isolation
- ✓ Team Autonomy
- ✓ Technology Flexibility

4 CORE DATA STORES

Users DB

- user_id (PK)
- name
- phone
- email
- role (rider/driver)
- ...

Trips DB

- trip_id (PK)
- rider_id (FK)
- driver_id (FK)
- status
- pickup_loc
- drop_loc
- fare
- started_at
- ended_at
- ...

Drivers DB

- driver_id (PK)
- user_id (FK)
- vehicle_no
- vehicle_type
- is_online
- rating
- ...

Locations (Redis)

- driver_id → geo_location
- driver_id → status
- active_trip → details
- ...

- ★ Realtime data kept in Redis for fast read & write

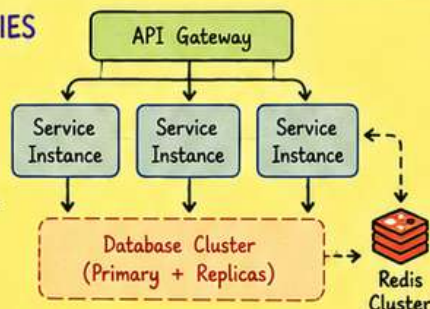
Object Storage (S3)

- profile_pics
- documents
- insurance
- receipts
- ...

- ★ Files, images & docs stored in Object Storage

5 SCALABILITY STRATEGIES

- ✓ Horizontal scaling of services
- ✓ Database read replicas
- ✓ Caching with Redis
- ✓ Asynchronous processing via Kafka
- ✓ CDN for static content
- ✓ Auto scaling groups in cloud



6 DESIGN GOALS ACHIEVED

- ✓ High Availability
- ✓ Low Latency
- ✓ Fault Tolerance
- ✓ Data Consistency
- ✓ Security
- ✓ Cost Effective
- ✓ Great User Experience

NOTE

This architecture can handle millions of users and thousands of rides per second!

KEY TAKEAWAY : Decouple services, use caching, async processing and scale horizontally to build a robust ride booking system!

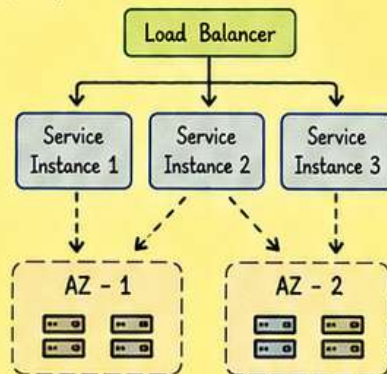
RELIABILITY & FAULT TOLERANCE

(Always Available, Even When Things Fail)



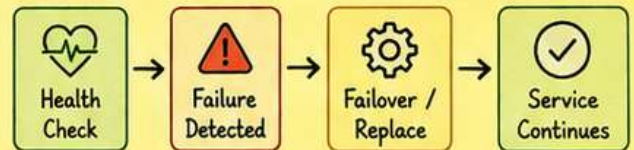
① HIGH AVAILABILITY (HA)

- System remains available 99.9% of the time
- No single point of failure
- Redundancy + Failover
- Multiple instances across zones



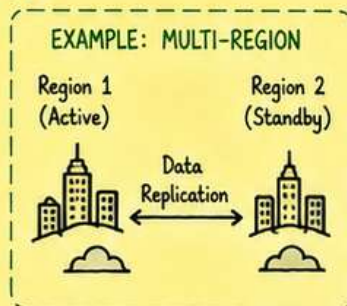
② FAULT TOLERANCE

- System continues working even if some parts fail
- Detect failures quickly
- Isolate faulty components
- Auto recovery / failover



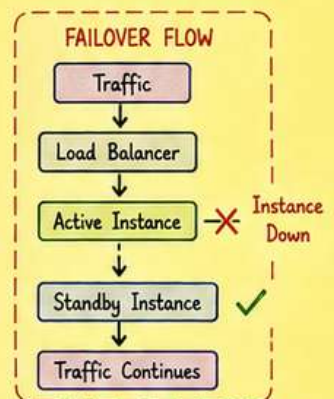
③ REDUNDANCY STRATEGIES

- ✓ Multi-Region Deployment
- ✓ Data Replication
- ✓ Backup & Restore
- ✓ Active-Passive / Active-Active
- ✓ Replicated Queues & DB
- ✓ Load Balancing



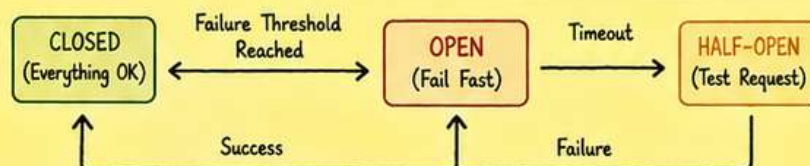
④ FAILOVER MECHANISM

- Monitor services continuously
- On failure, traffic auto-routes to healthy instance
- Use Health Check + Load Balancer
- DNS Failover for regional outages



⑤ CIRCUIT BREAKER PATTERN

- Prevent cascading failures
- If a service keeps failing, "OPEN" the circuit
- After timeout, try again (HALF-OPEN)



⑥ DATA CONSISTENCY

- Use ACID for critical transactions
- Eventual Consistency for non-critical (e.g., driver location updates)
- Idempotent APIs
- Use Versioning / Timestamps

CONSISTENCY LEVELS

Strong Consistency	★★★★★
Eventual Consistency	★★★☆☆
Session Consistency	★★★★☆
Monotonic Reads	★★★★☆

⑦ RETRY & TIMEOUT STRATEGY

- ✓ Use exponential backoff
- ✓ Set sensible timeout
- ✓ Retry only for idempotent operations
- ✓ Limit max retries

EXAMPLE (BACKOFF)

Attempt 1	→ Wait 1s
Attempt 2	→ Wait 2s
Attempt 3	→ Wait 4s
Max Retries	→ Give Up

⑧ MONITORING & ALERTING

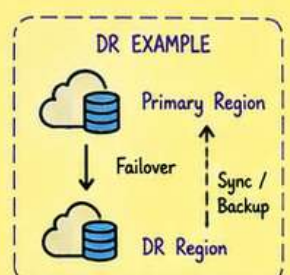
- Monitor key metrics (Availability, Latency, Error Rate)
- Set up alerts for anomalies
- Use Dashboards for real-time visibility
- Log aggregation & tracing

KEY METRICS

- Uptime %
- Response Time (P95, P99)
- Error Rate
- Throughput (RPS)
- Saturation / Resource Usage

⑨ DISASTER RECOVERY (DR)

- Regular backups
- RTO (Recovery Time Objective)
- RPO (Recovery Point Objective)
- Periodic DR drills / Testing
- Automated recovery scripts



KEY TAKEAWAY: Design for failure, expect the unexpected, and recover automatically. Reliability builds trust! ❤️

RIDE MATCHING SYSTEM

(Find the Best Driver for Every Ride)



1 MATCHING FLOW



Goal: Minimize ETA, Maximize Success Rate, Fair to Drivers

2 MATCHING CRITERIA

- ★ Distance to rider (proximity)
- ★ Driver Availability & Status
- ★ Driver Rating & Cancellation Rate
- ★ Vehicle Type Match
- ★ Current Load / Active Trips
- ★ Historical Performance
- ★ Surge Multiplier & Incentives
- ★ Fairness (avoid bias to any driver)

Smart matching = Happy Riders + Happy Drivers + High Success Rate

3 DRIVER STATE TRANSITIONS



Heartbeat updates ensure real-time status accuracy

4 NEARBY DRIVER SEARCH

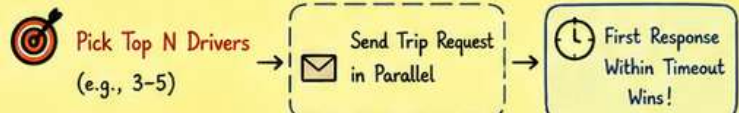


- Use Geo-Spatial Index (GeoHash)
- Search in expanding radius
- Partition by zones / grids
- Only online & active drivers
- Real-time location from Redis

Tech: GeoHash / H3 Index + Redis Geospatial

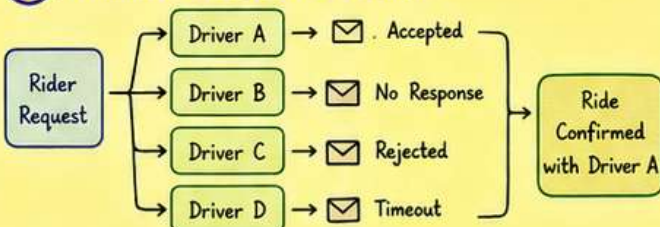
5 RANKING ALGORITHM (EXAMPLE)

$$\text{Score} = w_1 * \text{Distance} + w_2 * \text{ETA} + w_3 * \text{Rating} + w_4 * \text{CancelRate} + w_5 * \text{LoadFactor} + w_6 * \text{SurgeBoost}$$



Weights ($w_1 \dots w_6$) are tuned using ML & A/B Testing

6 PARALLEL REQUEST STRATEGY



7 HANDLING EDGE CASES

- No drivers available -> increase radius / show wait time
- All drivers rejected -> try again with better incentives
- Driver goes offline -> reassign to next best driver
- High demand -> apply surge & notify user
- Poor network -> store request & retry

Always have a fallback plan!

8 KEY POINTS

- ✓ Fast & smart matching reduces rider wait time
- ✓ Fair distribution keeps drivers happy
- ✓ Real-time data + good algorithm = high success rate
- ✓ Continuously learn & optimize using data



METRICS TO TRACK

- Matching Success Rate (%)
- Average Rider Wait Time
- Driver Response Rate
- Cancellation Rate
- Reassignment Rate
- Surge Utilization



GOAL: Right Driver, Right Time, Right Place = Happy Rides Every Time! ❤️

RIDE BOOKING SYSTEM

(End-to-End Flow & Key Modules)



① END-TO-END FLOW



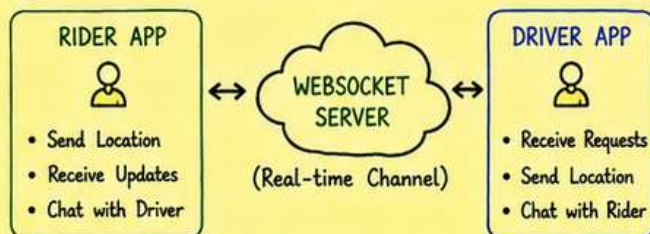
★ Real-time updates at every step for Rider & Driver via notifications, location & status. ❤️

② CORE MODULES

- ✓ User Service — Rider & Driver management
- ✓ Matching Service — Match best driver
- ✓ Trip Service — Manage trip lifecycle
- ✓ Location Service — Real-time GPS tracking
- ✓ Pricing Service — Fare calculation & surge
- ✓ Payment Service — Secure payments
- ✓ Notification Service — Alerts & updates
- ✓ Review Service — Ratings & feedback
- ✓ Admin Service — Dashboard & operations

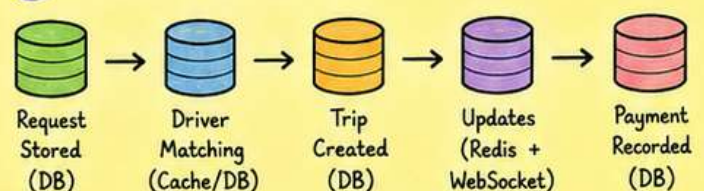
💡 Modular design → Easy to scale, deploy & maintain! ✨

③ REAL-TIME COMMUNICATION



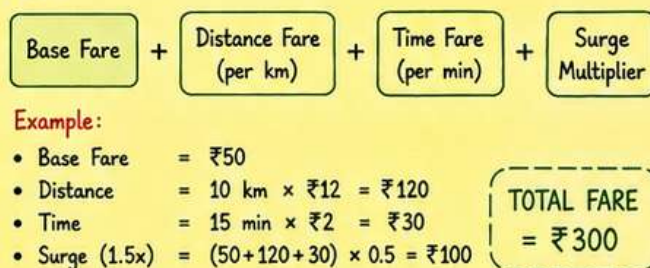
⚡ Low latency, bi-directional communication for smooth experience.

④ DATA FLOW OVERVIEW

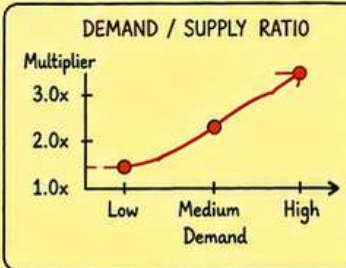


🎯 Use Redis for fast reads, WebSocket for live updates, and DB for consistency. ❤️

⑤ FARE CALCULATION



⑥ SURGE PRICING LOGIC



FACTORS

- High demand
- Low driver availability
- Time (Peak hours)
- Location (Hotspots)
- Events / Weather

★ Dynamic multiplier ensures balance between supply & demand.

⑦ LOCATION TRACKING



📍 Use Redis GEO commands for fast nearby search & distance calculation.

⑧ PAYMENT FLOW



SUPPORTED METHODS

- Cards (Debit/Credit)
- UPI / Wallets
- Net Banking
- Cash (if supported)
- Corporate Accounts

⑨ KEY TAKEAWAYS

- ★ Real-time matching is the heart of ride booking.
- ★ Location + Data + Algorithms = Great Experience.
- ★ Scalability, Reliability & Security are must!
- ★ Happy Riders → Happy Drivers → Happy Business ❤️



🎯 **ULTIMATE GOAL:** Right Driver, Right Time, Right Price, Right Place = **PERFECT RIDE!** 😊

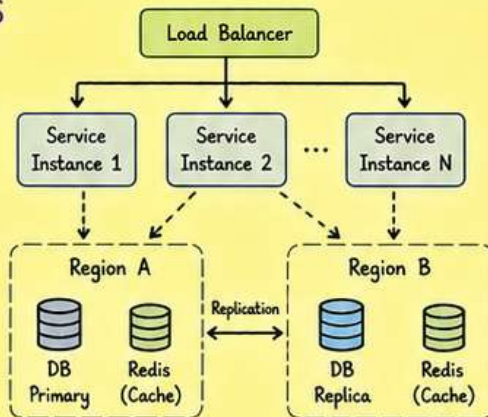
RIDE BOOKING SYSTEM

(Scaling, Optimization & Advanced Topics)



1 SCALING STRATEGIES

- ✓ Horizontal scaling of services
- ✓ Auto scaling based on demand
- ✓ Load balancing across regions
- ✓ Database read replicas
- ✓ Caching (Redis) for hot data
- ✓ CDN for static assets
- ✓ Asynchronous processing for notifications, emails, receipts



★ Scale stateless services horizontally & keep data layer highly available. ❤️

2 OPTIMIZATIONS

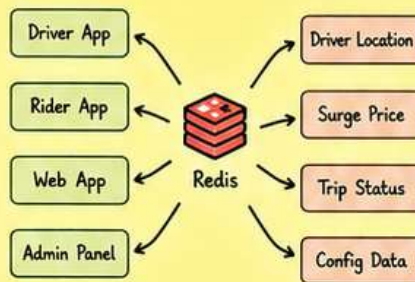
- ★ Use geohash index for nearby search
- ★ Use connection pooling for DB
- ★ Cache driver location & trip status
- ★ Compress API responses
- ★ Use pagination for lists
- ★ Batch similar notifications
- ★ Remove unused data & archive old trips
- ★ Monitor & profile for bottlenecks
- ★ Use efficient algorithms for matching & routing

💡 Optimize for speed, cost & user experience. ❤️

3 CACHING STRATEGY

WHAT TO CACHE?

- ❑ Driver Location (TTL: 30s)
- ❑ Surge Prices (TTL: 60s)
- ❑ Popular Locations
- ❑ Cancellation Reasons
- ❑ Configuration / App Settings



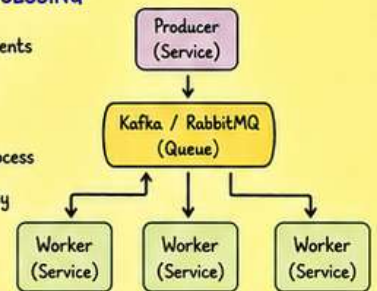
★ Reduce DB load, improve response time & handle high read traffic efficiently. ❤️

4 QUEUE & ASYNC PROCESSING

- ✓ Use Kafka / RabbitMQ for events
- ✓ Separate queues for different workloads
- ✓ Worker services consume & process
- ✓ Improves reliability & scalability

EXAMPLES OF EVENTS

- TripCreated
- PaymentCaptured
- NotificationSend
- DriverLocationUpdated
- RideCompleted

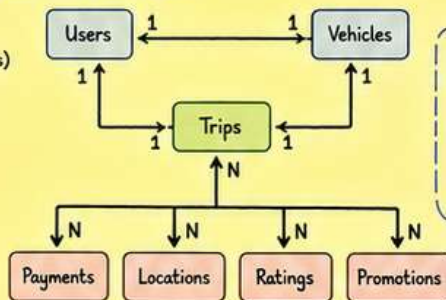


⚡ Async = Resilient, Scalable & Fault-Tolerant System. ❤️

5 DATABASE DESIGN OVERVIEW

Core Tables

- Users (Riders & Drivers)
- Vehicles
- Trips
- Locations
- Payments
- Promotions
- Ratings & Reviews
- Notifications
- Support Tickets



★ Proper indexing & relationships ensure fast queries & scalability. ❤️

6 SECURITY CONSIDERATIONS

- 🛡️ HTTPS everywhere
- 🛡️ JWT for authentication
- 🛡️ Role-based access control
- 🛡️ Encrypt sensitive data
- 🛡️ PCI-DSS for payments
- 🛡️ Rate limiting & throttling
- 🛡️ Input validation & sanitization
- 🛡️ Audit logs & monitoring
- 🛡️ Regular security updates



★ Security builds trust & protects users and business. ❤️

7 MONITORING & ALERTING

- 📈 Monitor key metrics
- 🔔 Set up alerts for anomalies
- 👁️ Dashboards for real-time visibility
- 📄 Log aggregation & tracing
- 🔍 Health checks for all services

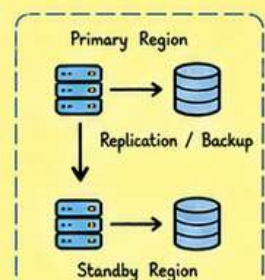
KEY METRICS

- ✓ Request Rate (RPS)
- ✓ Error Rate (%)
- ✓ Average Response Time
- ✓ DB / Cache Hit Ratio
- ✓ Active Trips
- ✓ Surge Multiplier
- ✓ Driver Availability

💡 Observe, Analyze & React Quickly! ❤️

8 DISASTER RECOVERY (DR)

- ✓ Multi-region deployment
- ✓ Regular backups (DB, Files)
- ✓ Automated failover
- ✓ RPO & RTO defined
- ✓ Periodic DR drills & testing
- ✓ Data replication & snapshots



FINAL GOAL: Seamless experience for riders, growth for drivers, and efficiency for the business! ❤️

RIDE BOOKING SYSTEM

SUMMARY CHEAT SHEET

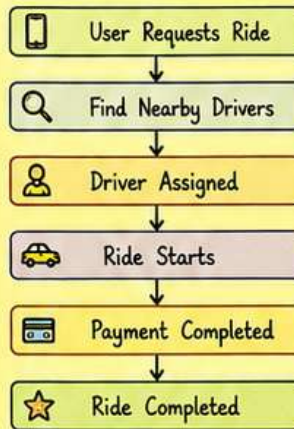


1 KEY COMPONENTS

- 📱 Client Apps — Rider, Driver, Web
- 🔒 API Gateway — Auth, Rate Limit, Routing
- 📦 Microservices — Independent, Single Responsibility
- 💾 Data Stores — MySQL, Redis, S3
- 🔄 Message Queue — Kafka / RabbitMQ
- 📊 Analytics & Monitoring — Metrics, Logs, Alerts

★ Decouple, Scale & Observe! ❤️

2 CORE FLOW



🎯 Right Driver, Right Time, Right Price, Right Place! ❤️

3 DESIGN GOALS

- ✓ High Availability (99.9%+)
- ✓ Low Latency & Fast Matching
- ✓ Scalability (Horizontal & Vertical)
- ✓ Fault Tolerance & Resilience
- ✓ Data Consistency & Reliability
- ✓ Security & Privacy
- ✓ Cost Effective
- ✓ Great User Experience ❤️

NOTE

This architecture can handle millions of users and thousands of rides per second! 😊

4 TECHNOLOGY STACK

- 🖥️ Backend : Java / Kotlin / Node.js
- 💾 Database : MySQL / PostgreSQL
- 📦 Cache : Redis
- 🔄 Message Queue : Kafka / RabbitMQ
- 📁 Storage : AWS S3 / GCP Cloud Storage
- 🏗️ Infra : Docker, Kubernetes
- 🔥 Monitoring : Prometheus, Grafana

★ Modern, Scalable & Cloud Native! ❤️

5 RELIABILITY PATTERNS

- 🛡️ Redundancy & Failover
- 🛡️ Circuit Breaker
- 🔄 Retry with Backoff
- 🛡️ Idempotency
- 📬 Dead Letter Queue
- 🛡️ Health Checks
- ⏰ Timeouts & Fallbacks

⚡ Expect Failure, Design for Recovery! ❤️

6 KEY DATA STORES

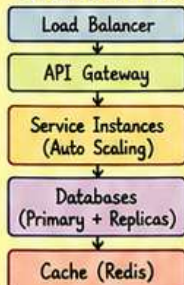
- 🗄️ Users DB — Riders, Drivers, Profiles
- 🗄️ Trips DB — Trips, Status, Timeline
- 🗄️ Drivers DB — Driver Info, Vehicle, Docs
- 🗄️ Locations (Redis) — Geo, Hotspots, Live Data
- 🗄️ Object Storage (S3) — Docs, Images, Receipts
- 🗄️ Cache (Redis) — Sessions, Config, Hot Data

★ Right Data, Right Store, Right Performance! ❤️

7 PERFORMANCE & SCALING

- Auto scaling with load
- Read replicas for DB
- CDN for static content
- Connection pooling
- Caching at multiple layers (Redis)
- Async processing via queues
- Optimize DB queries & indexes
- Monitor, measure & iterate

SCALING LAYERS



8 MONITORING & ALERTING

- 📈 Monitor Key Metrics
- 🔔 Alerts for Anomalies
- 👁️ Dashboards for Insights
- 📄 Log Aggregation & Tracing
- 🏥 Health Checks
- 🕒 SLOs & Error Budgets

★ Observe Everything, Fix Issues Early! ❤️

9 METRICS TO TRACK

- Matching Success Rate (%)
- Average Wait Time
- Driver Response Time
- Trip Completion Rate
- Cancellation Rate
- Surge Multiplier
- Driver Utilization
- System Latency (P95/P99)
- Error Rate & Availability



10 KEY TAKEAWAYS

- ★ Microservices + Async + Caching = High Performance
- ★ Resilience Patterns = Reliable System
- ★ Data Driven Decisions = Better User Experience
- ★ Continuous Optimization = Long Term Success ❤️



REMEMBER:

A Great Ride Booking System is not just about code, it's about creating a seamless & trusted experience for millions of riders & drivers! ❤️



FINAL GOAL: HAPPY RIDERS + HAPPY DRIVERS = **SUCCESSFUL BUSINESS!** ❤️

15